

Eingereicht von
Tobias Rafetseder
12306137

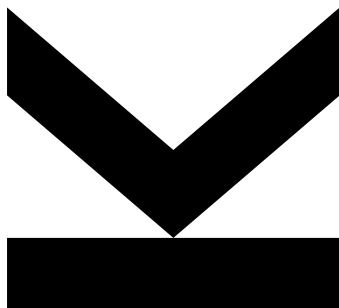
Angefertigt am
Institut für
Wirtschaftsinformatik -
Data and Knowledge
Engineering

Betreuer
o. Univ.-Prof. Dipl.-Ing. Dr.
Michael Schrefl

Mitbetreuung
Dipl.-Ing.
Sebastian Windsperger

Mai 2026

Ein ASP-basierter Ansatz zur Lösung des Nurse Scheduling Problems mit Clingo



Bachelorarbeit

zur Erlangung des akademischen Grades

Bachelor of Science

im Bachelorstudium

Wirtschaftsinformatik

Kurzfassung

Space for your abstract.

Kurzfassung

Platz für deine Kurzfassung.

Inhaltsverzeichnis

Kurzfassung	ii
Kurzfassung	iii
1 Einleitung	1
1.1 Einleitung	1
1.2 Ziel	1
2 Example	3
2.1 How to Start?	3
3 Grundlagen	6
3.1 Nurse Scheduling Problem	6
3.2 Answer Set Programming	7
3.2.1 Syntax und Semantik	7
3.2.2 Answer Sets als Lösungskonzept	7
3.2.3 Verarbeitung und Werkzeuge	8
3.2.4 Anwendungsgebiete	8
3.3 Clingo	8
4 Vergleich bestehender Systeme	10
4.1 Das ASP-basierte Nurse Scheduling System an dem Krankenhaus der Universität von Yamanashi	10
5 Conclusion and Outlook	11
Literatur	12
Anhang A An Appendix	13

Kapitel 1

Einleitung

1.1 Einleitung

Die Erstellung von Dienstplänen in Krankenhäusern stellt eine der komplexesten organisatorischen Aufgaben im Gesundheitswesen dar. In den Abteilungen müssen zahlreiche Anforderungen gleichzeitig berücksichtigt werden: Qualifikationen und Verfügbarkeiten des Pflegepersonals, gesetzliche Arbeitszeitregelungen, sowie individuelle Präferenzen der Mitarbeitenden. Kurzfristige Personalausfälle erhöhen die Komplexität zusätzlich und machen eine rein manuelle Planung zunehmend ineffizient. Die händische Erstellung von Dienstplänen ist nicht nur zeitaufwendig, sondern auch fehleranfällig und bietet wenig Skalierbarkeit bei wachsenden Anforderungen. Die Informatik stellt seit Jahrzehnten formale Methoden zur Lösung derartiger kombinatorischer Optimierungsprobleme bereit. Das sogenannte Nurse Scheduling Problem (NSP) ist in der wissenschaftlichen Literatur gut etabliert und wurde mit einer Vielzahl von Ansätzen untersucht, zum Beispiel mit Constraint Programming. In der praktischen Anwendung mangelt es jedoch häufig an Lösungen, die gleichermaßen flexibel gegenüber abteilungsspezifischen Anforderungen, transparent in ihrer Modellierung und nachhaltig wartbar sind. Answer Set Programming (ASP) bietet in diesem Kontext einen vielversprechenden Ansatz. Als deklaratives Paradigma der Logikprogrammierung ermöglicht ASP eine präzise und gut lesbare Modellierung von Constraints und Optimierungszielen, ohne dass problemspezifische Suchalgorithmen explizit implementiert werden müssen. Mit Clingo, einem der leistungsfähigsten und in der Forschung weitverbreitetsten ASP-Solver, steht ein effizientes Werkzeug zur Verfügung.

1.2 Ziel

Ziel dieser Bachelorarbeit ist die Entwicklung eines ASP-basierten Ansatzes zur automatisierten Erstellung und Optimierung von Dienstplänen in der Augenabteilung eines Krankenhauses, sowie dessen Umsetzung als vollständige Fullstack-Webanwendung. Die Applikation umfasst ein benutzerorientiertes Frontend zur Verwaltung von Personal, Stationen und Kompetenzen sowie ein Backend, das die Optimierungslogik mittels Clingo ausführt und die generierten Dienstpläne strukturiert zurückliefert. Sowohl harte Constraints, wie Mindestbesetzung und Qualifikationsanforderungen, als auch weiche Constraints, wie eine gleichmäßige Verteilung der Zuteilungen zu Stationen, werden dabei

formal modelliert und automatisiert gelöst. Die Arbeit untersucht damit, inwieweit ASP als Grundlage für eine praxistaugliche, vollständig integrierte Planungssoftware im klinischen Umfeld eingesetzt werden kann.

Kapitel 2

Example

2.1 How to Start?

This is an example for another chapter in your thesis. It should primarily show you some basic $\LaTeX$ tricks. For instance, Abschnitt 1.1 references to your motivation. You can also embed figures, tables, etc. in your work. Abbildung 2.1 is an example for a figure (i.e. photos, diagrams, and other artwork). Note that figures (just like tables and code listings, see Tabelle 2.1 and Listing 2.1) are floating elements. They are either placed at the top or bottom of a page (or sometimes stand on their own page), but not in between your text. You can influence their placement with the placement modifiers “t”, “b”, and “p”. Bottom placement (“b”) is sometimes more tricky, as $\LaTeX$ does not consider this nice in some situations. You can convince $\LaTeX$ to honor your placement expectation with “!b” then. Please be aware that you typically do not want to place a float at the top of a chapter title page. Since you do not place figures/tables in between your running text, you always need to reference them by their label (e.g. Abbildung 2.1 or Tabelle 2.1).

You will often reference and quote other works. The source of these citations needs to be marked with the `\cite{}` command. There are different ways to cite the work of others. These are a few examples:

- The Tor directory authority moria1 shows a voting behavior for the HS-Dir flag that significantly deviates from that of other directory authorities **bib:2021-hoeller-iiwas**. Be aware that the `\cite` command is part of the sentence and, hence, comes before the period.

Tabelle 2.1: A table. Be aware that tables have their caption above the table. Captions may span multiple lines

Option	Description
phdthesis	PhD thesis
mathesis	Master’s thesis
diplomathesis	Diploma thesis
bathesis	Bachelor’s thesis
seminarreport	Seminar report
techreport	Technical report



Abbildung 2.1: A figure. Be aware that figures have their caption below the artwork. Captions may span multiple lines

```

1 for (int i = 0; i < 100; ++i) {
2     printf("%3d\n", i);
3 }

```

Listing 2.1: A code listing. Code listings usually have their caption below the code. Captions may span multiple lines

- **bib:2015-roland-thesis-book bib:2015-roland-thesis-book** proposes a novel attack concept against NFC secure elements in mobile phones.
- **bib:2013-roland-woot bib:2013-roland-woot** uncovered a flaw in legacy-support of the MasterCard contactless payment protocols that allows an attacker to clone certain credit cards.
- **bib:2021-hoeller-foci bib:2021-hoeller-foci** designed an experiment to measure the usage of Tor V3 onion services in a privacy-conscious way. Here we used “et al.” because the cited work has more than two authors (\citeauthor will automatically take care of this).
- **bib:2021-hoeller-iiwas bib:2021-hoeller-iiwas** conclude:

Ultimately, the high fluctuations in the hidden service directory were caused by a mixture of several issues. First the changed voting behavior of three directory authorities reduced the amount of obtainable votes to six. If any of the remaining six relays went offline – which tends to happen during ongoing DOS attacks – relays needed to obtain five out of five available votes. So any individual measurement failure regarding either bandwidth or uptime led to a withdrawn HSDir flag.

- You sometimes also paraphrase from multiple sources. For that pur-

pose, the `\cite` command accepts a list of multiple comma-separated references. Do not add spaces in between them. Various analyses of the Tor network have been performed recently **bib:2021-hoeller-foci**; **bib:2021-hoeller-iiwas**.

Kapitel 3

Grundlagen

3.1 Nurse Scheduling Problem

Das *Nurse Scheduling Problem* (NSP) beschreibt die Zuordnung von Arbeitsschichten für Pflegekräfte innerhalb eines bestimmten Planungszeitraums. Ziel ist es, die Abfolge der Schichten für jede Pflegekraft so festzulegen, dass eine kontinuierliche und angemessene Patientenversorgung gewährleistet wird. Die Erstellung solcher Dienstpläne stellt eine komplexe und zeitaufwendige Aufgabe dar. Gleichzeitig beeinflussen Dienstpläne mit unterschiedlichen Schichten das Stressniveau der Pflegekräfte sowie deren soziale Beziehungen und Familienleben erheblich (Oughalime et al., 2008).

Das Nurse Scheduling Problem umfasst die Zuweisung Schichten unter Berücksichtigung verschiedener Nebenbedingungen. Dabei muss sichergestellt werden, dass jederzeit ausreichend Personal vorhanden ist, ohne unnötige Überbesetzung zu verursachen. Die Dienstplanung erfordert daher umfangreiches Wissen und Erfahrung und wird in vielen Krankenhäusern manuell durch Stationsleitungen oder andere verantwortliche Personen durchgeführt. Ein geeigneter Dienstplan soll nicht nur die täglichen Schichtanforderungen erfüllen, sondern auch Fairness zwischen den Pflegekräften gewährleisten und ihre Gesundheit sowie ihr Privatleben möglichst wenig beeinträchtigen (Oughalime et al., 2008).

Zur Lösung des Nurse Scheduling Problems wurden verschiedene mathematische und heuristische Verfahren entwickelt. In der Literatur werden unter anderem Goal Programming, Simulated Annealing und Tabu Search eingesetzt. Ziel dieser Verfahren ist es, sowohl die Anforderungen des Krankenhauses als auch die Präferenzen der Pflegekräfte zu berücksichtigen (Oughalime et al., 2008). Der in der vorliegenden Arbeit betrachtete Ansatz basiert auf einer ASP-basierten Lösung.

Hard Constraints müssen in jedem Fall erfüllt werden, da der Dienstplan ansonsten als unzulässig gilt. Dazu gehört beispielsweise die Sicherstellung einer Mindestanzahl an Pflegekräften pro Schicht und Tag. Außerdem darf eine Pflegekraft pro Tag nur einer Schicht zugewiesen werden. *Soft Constraints* repräsentieren die Wünsche und Präferenzen der Pflegekräfte. Diese sollen möglichst erfüllt werden, sind jedoch nicht zwingend erforderlich. Dazu gehört beispielsweise die bevorzugte Anzahl an Arbeitstagen pro Planungszeitraum. Außerdem sollen ungünstige Schichtfolgen vermieden werden (Oughalime et al., 2008).

3.2 Answer Set Programming

Answer Set Programming (ASP) ist ein deklarativer Ansatz zur Modellierung und Lösung von Such- und Optimierungsproblemen. Es entstand aus der Schnittmenge mehrerer Forschungsgebiete, insbesondere der Wissensrepräsentation, der Logikprogrammierung und der Constraint-Satisfaction, und vereint deren Stärken in einem einheitlichen Framework. ASP erlaubt es, Probleme deklarativ zu formulieren, ohne dass der Programmierer den Suchprozess selbst steuert.

3.2.1 Syntax und Semantik

Die grundlegenden Bausteine eines ASP-Programms sind Atome, Literale und Regeln. Atome sind elementare Aussagen mit einem Wahrheitswert, Literale sind entweder Atome oder deren Negation. Regeln haben die allgemeine Form:

```
1 a ← b1, . . . , bm, not c1, . . . , not cn
```

Dabei bezeichnet a , also die linke Seite der Regel den Regelkopf, und die rechte Seite den Regelrumpf. Die Regel ist so zu interpretieren: Atom a gilt, sofern alle b_m ableitbar sind und keines der c_n ableitbar ist. Der Operator **not** ist nicht als klassische Negation zu verstehen, sondern als Default-Negation, die die Nicht-ableitbarkeit eines Atoms ausdrückt. Als Beispiel:

```
1 broken ← lightning, not lightning_rod
2 light_on ← power_on, not broken
```

Hier leuchtet die Lampe, wenn der Strom an ist und es keinen Grund gibt anzunehmen, dass sie kaputt ist. Ist nichts über einen Defekt bekannt, gilt $\langle \text{not broken} \rangle$, und die Lampe leuchtet. Wenn $\langle \text{lightning} \rangle$ als Fakt hinzukommt, wird $\langle \text{broken} \rangle$ ableitbar, und $\langle \text{not broken} \rangle$ gilt nicht mehr, also leuchtet die Lampe nicht mehr. Da ASP eng mit der nichtmonotonen Logik verwandt ist, können bereits gezogene Schlussfolgerungen zurückgezogen werden, sobald neue Fakten oder Regeln hinzugefügt werden. Die Default-Negation ist dabei der zentrale Mechanismus, der dieses Verhalten ermöglicht.

3.2.2 Answer Sets als Lösungskonzept

Die Semantik von ASP-Programmen basiert auf dem Begriff des Answer Sets, auch früher unter dem Namen *stable models* bekannt, der auf Gelfond und Lifschitz, 2000 zurückgeht. Ein Answer Set ist eine Menge von Atomen, die in sich konsistent ist und bei der jedes enthaltene Atom durch eine Regelanwendung begründet werden kann. Ein Programm kann dabei mehrere, genau eines oder kein Answer Set besitzen, wobei jedes Answer Set einer gültigen Lösung des modellierten Problems entspricht (Brewka et al., 2011). Diese Mehrwertigkeit macht ASP besonders geeignet für Probleme, bei denen mehrere gleichwertige Lösungen existieren können, wie zum Beispiel bei der Dienstplanung.

3.2.3 Verarbeitung und Werkzeuge

Die Ausführung eines ASP-Programms erfolgt typischerweise in zwei Schritten. Zunächst übersetzt ein sogenannter *Grounder* das Programm bestehend aus Prädikaten in eine äquivalente aussagenlogische Form, indem alle Variablen durch konkrete Konstanten ersetzt werden. Anschließend berechnet ein *Solver* die Answer Sets des resultierenden Programms. Bekannte Systeme sind unter anderem CLASP, DLV und SMODELs. Moderne Solver nutzen dabei Techniken aus dem SAT-Solving, die sicherstellt, dass jedes wahre Atom tatsächlich ableitbar ist (Brewka et al., 2011).

3.2.4 Anwendungsgebiete

Aufgrund seiner Ausdruckstärke und Flexibilität findet ASP Anwendung in einer Vielzahl von Anwendungsbereichen. Industriell wurde es beispielsweise zur automatisierten Schichtplanung im Containerhafen Gioia Tauro eingesetzt, wo ein ASP-basiertes System die manuelle Planung von mehreren Stunden auf wenige Minuten reduzierte und gleichzeitig die Planqualität verbesserte. Weitere Anwendungsfelder umfassen Entscheidungsunterstützungssysteme, die Analyse biologischer Netzwerke, phylogenetische Systematik sowie klassische kombinatorische Probleme wie den Hamiltonkreis (Brewka et al., 2011).

3.3 Clingo

Clingo ist ein modernes ASP-System zur Berechnung von *stable models* und erweitert klassische ASP-Systeme um das Konzept des Multi-Shot Solving. Während frühere Systeme einem einmaligen („one-shot“) Ablauf folgen, der aus Grounding und anschließendem Solving besteht, ermöglicht Clingo die Verarbeitung sich kontinuierlich verändernder Logikprogramme innerhalb eines laufenden Prozesses. Dadurch können Programme schrittweise erweitert, angepasst oder deaktiviert werden, ohne den Grounder oder Solver neu starten zu müssen (Gebser et al., 2018).

Ein zentrales Merkmal von Clingo ist die Integration von Grounder und Solver innerhalb eines einzigen Systems. Frühere Versionen von Clingo bestanden aus der Kombination des Grounders gringo und des Solvers clasp. Mit der Einführung des Multi-Shot Solving wurde Clingo jedoch um Mechanismen erweitert, die einen flexiblen und zustandsbasierten Lösungsprozess erlauben. Dabei bleibt der interne Zustand des Systems erhalten und kann durch verschiedene Operationen verändert werden. Dies reduziert Redundanzen beim erneuten Grounding und ermöglicht gleichzeitig die Wiederverwendung bereits gelernter Informationen des Solvers (Gebser et al., 2018).

Ein weiterer wichtiger Bestandteil von Clingo ist die bereitgestellte Programmierschnittstelle (API). Diese erlaubt die Kontrolle des Grounding- und Solving-Prozesses aus einer prozeduralen Programmiersprache heraus. Clingo stellt hierfür eine C-API bereit, sowie Bindings für Python und Lua (Gebser et al., 2018). Besonders relevant für diese Arbeit ist die Integration von Python,

da sie eine flexible Steuerung komplexer Reasoning-Prozesse innerhalb von Clingo ermöglicht.

Die Integration von beispielsweise Python erfolgt über das *Directive* `#script(python)`. Innerhalb eines solchen Skriptblocks kann eine *main*-Funktion definiert werden, die Zugriff auf ein Kontrollobjekt erhält. Über dieses Objekt können Subprogramme gezielt *gegrounded* und *Solving*-Aufrufe gestartet werden. Beispielsweise kann mit `prg.ground(...)` ein bestimmtes Teilprogramm instanziiert und mit `prg.solve()` die Berechnung von *Answer Sets* ausgelöst werden. Dadurch wird die deklarative Wissensrepräsentation von ASP mit prozeduraler Steuerungslogik kombiniert (Gebser et al., 2018).

Kapitel 4

Vergleich bestehender Systeme

Im Zuge der Entwurfsphase des NSP-Solvers wurde eine Literaturrecherche zu bestehenden Systemen und Ansätzen durchgeführt. Auf ähnliche ASP-basierte Implementierungen im Gesundheitssektor wird in den nächsten Punkten eingegangen.

4.1 Das ASP-basierte Nurse Scheduling System an dem Krankenhaus der Universität von Yamanashi

An dem Krankenhaus der Universität von Yamanashi wurde ein *Nurse Scheduling System* mit Clingo umgesetzt. Dabei wurden Einschränkungen wie *Workdays*, *Weekly Rest Days*, *Requested Shifts*, *Average Working Hours*, *Consecutive Workdays*, *Shift Frequency* sowie *Daily Staffing* berücksichtigt. Das dort entwickelte System wird als Web-Service bereitgestellt, was Pflegeleitungen erlaubt, über den Browser Dienstpläne zu erstellen und zu modifizieren. Fokus der Arbeit war das Erstellen von komplexen Dienstplänen alle vier Wochen.

Die Applikation unterstützt nicht nur die automatische Erstellung von Dienstplänen, sondern auch eine interaktiven Nachbearbeitung und Anpassung bestehender Pläne. Pflegeleitungen können einzelne Schichten manuell ändern, gewünschte oder unerwünschte Dienste definieren sowie bestimmte Zuweisungen fixieren, damit diese bei späteren Optimierungsschritten erhalten bleiben. Dies ist besonders relevant, da in der Praxis häufig kurzfristige Änderungen durch Krankheitsausfälle oder andere unvorhersehbare Ereignisse auftreten und somit ein *Rescheduling* erforderlich wird.

Ein weiterer zentraler Aspekt des Systems ist die Minimierung unnötiger Änderungen an bereits bestehenden Dienstplänen. Dadurch soll der Aufwand für die Überprüfung der Pläne reduziert und vermieden werden, dass Pflegekräfte ständig über größere Änderungen informiert werden müssen. Zur Umsetzung dieses Ansatzes wird die Methode *Large Neighborhood Prioritized Search (LNPS)* eingesetzt. Im Gegensatz zu klassischen Verfahren der Minimal Perturbation werden dabei bestehende Schichtzuweisungen bevorzugt beibehalten, ohne sie jedoch vollständig zu fixieren. Dadurch bleibt die Lösung flexibel genug, um weiterhin Optimierungen durchführen und neue Randbedingungen berücksichtigen zu können. Die Priorisierung wird mithilfe von Heuristiken des ASP-Solvers Clingo umgesetzt.

Kapitel 5

Conclusion and Outlook

Space for your summary, central conclusions, and an outlook on potential future work.

Literatur

- Brewka, G., Eiter, T., & Truszczyński, M. (2011). Answer set programming at a glance. *Communications of the ACM*, 54(12), 92–103. <https://doi.org/10.1145/2043174.2043195>
- Gebser, M., Kaminski, R., Kaufmann, B., & Schaub, T. (2018, 20. März). *Multi-shot ASP solving with clingo*. arXiv: 1705.09811 [cs]. <https://doi.org/10.48550/arXiv.1705.09811>
- Gelfond, M., & Lifschitz, V. (2000). The Stable Model Semantics For Logic Programming. *Logic Programming*, 2.
- Oughalime, A., Ismail, W. R., & Liong, C. Y. A tabu search approach to the nurse scheduling problem [Cited by: 17]. In: 1. Cited by: 17. 2008. <https://doi.org/10.1109/ITSIM.2008.4631604>

Anhang A

An Appendix

Space for an appendix. You can have more than one appendix chapter. Appendices are, of course, optional.